Guía Proyecto 3 (M.U.M.N.K8s)

1. Dinámica

- **Modalidad:** evaluación presencial por bloques de 5 estudiantes.
- **Regla:** todo el sistema debe estar **completamente desplegado y funcionando** antes de iniciar el bloque del grupo. No se permite compilar, desplegar o levantar servicios durante el turno.

2. Indicaciones previas para el estudiante

Antes del ingreso de su grupo, cada estudiante debe tener preparado en su computadora:

1. **Pestañas de terminal:** conectadas a Google Cloud Platform (`gcloud` / `kubectl` configurado hacia GKE).
2. **Pestaña de navegador 1:** interfaz de **Locust** lista para iniciar y mostrar pruebas de carga.
3. **Pestaña de navegador 2:** catálogo de imágenes del Container Registry **Zot** (HTTPS en VM de GCP).
4. **Pestaña de navegador 3:** dashboard de **Grafana** cargado.

3. Criterios de evaluación (orden de la rúbrica)

Parte 1: Implementación funcional

1.1 Arquitectura general, despliegue (Gateway API)

- **Comandos:**

```
kubectl get nodes -o wide
kubectl get gateway,httproute -n <namespace>
```

- **Verificaciones:**
  - Despliegue efectivo del cluster.
  - Exposición de servicios mediante **Kubernetes Gateway API** con la ruta **`/grpc-#carnet.`**

1.2 API REST en Rust

- **Comandos:**

```
kubectl get hpa,deployments -n <namespace>
```

- **Verificaciones:**
  - API desarrollada en Rust, funcionando y escalando.

1.3 Servicios en Go (API/gRPC Client, gRPC Server/Writers)

- **Comandos:**

```
kubectl logs -l app=api-rust --tail=10
```

- **Verificaciones:**
  - Comunicación entre servicios: Rust (API REST) ⟶ Go (gRPC Client) ⟶ Go (gRPC Server / Writer).
  - Manejo de concurrencia y comunicación gRPC correctos.

1.4 Arquitectura general, despliegue en Zot

- **Pantalla (navegador):** repositorio de imágenes publicado en la VM externa con **Zot**.
- **Verificaciones:**
  - Zot desplegado en una VM de GCP, fuera del clúster.
  - Almacenamiento y descarga de todas las imágenes Docker de los microservicios.

1.5 Message broker y consumidor RabbitMQ

- **Comandos:**

```
kubectl get pods -n <namespace>
```

- **Verificaciones:**
  - Uso obligatorio de **RabbitMQ** como broker de mensajería principal.

1.6 Almacenamiento (Valkey)

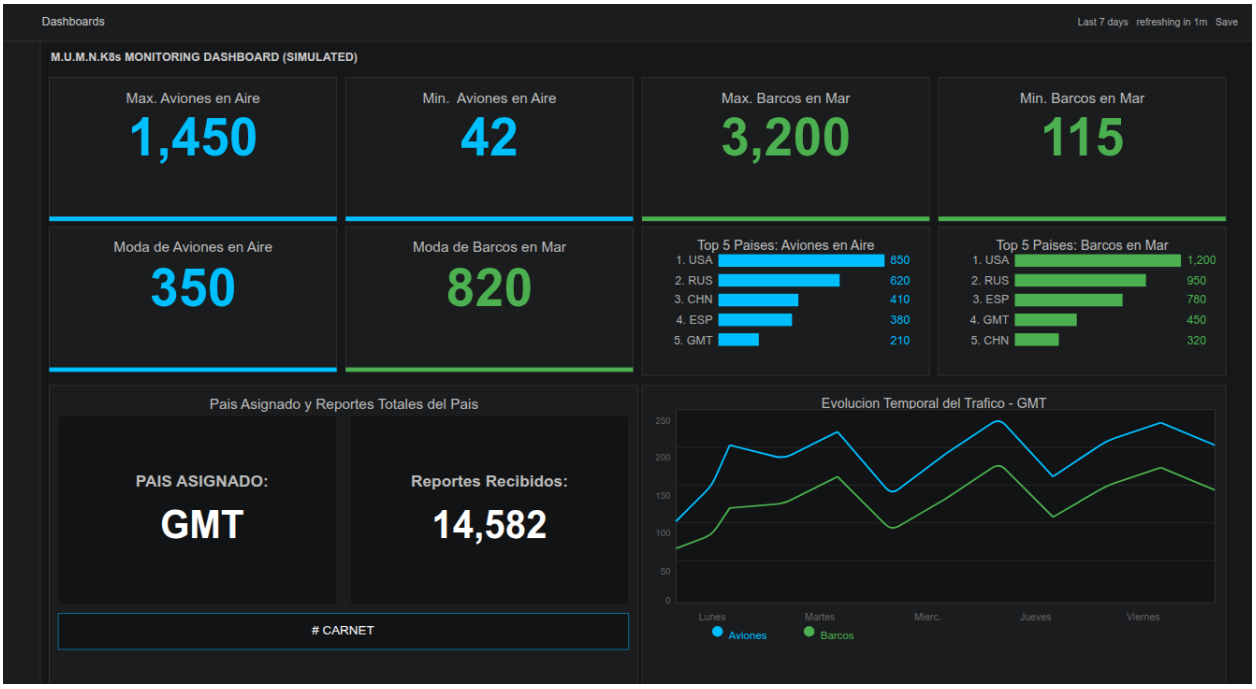
- **Comandos:**

```
# Inspeccionar persistencia en Valkey
kubectl exec -it <pod-valkey> -n <namespace> -- valkey-cli DBSIZE
```

- **Verificaciones:**
  - Operator desarrollado con Operator dentro del clúster GKE.
  - Custom Resource (CRD `ValkeyInstance`) definido y en uso, con reconciliación automática.
  - Persistencia correcta mediante PVC.

1.7 Grafana

- **Pantalla (navegador):** dashboard principal cargado en **Grafana**, desplegado por el Operator.
- **Verificaciones del evaluador:**
  - Cumple con lo pedido en el enunciado



1.8 Falco

- **Verificaciones:**
  - Falco instalado y operativo en el clúster de GKE.
  - **Simulación en vivo que dispare la alerta en tiempo real en los logs.**

2.1 Pruebas de carga, HPA y análisis de réplicas

- **Verificaciones:**
  - Pruebas comparativas con 1 y 2 réplicas para Go Writers y Valkey, documentadas y analizadas.

2.2 Manual técnico y guía de instalación

- **Verificaciones:**
  - Manual en formato Markdown.
  - Explica arquitectura, configuración de Gateway API, flujo gRPC/REST, RabbitMQ, HPA, uso del OCI Artifact en Zot y diseño del Operator.
  - Incluye el YAML de la regla personalizada de Falco y capturas de los logs con la alerta generada.
  - Guía de instalación clara y completa.

2.2 Respuestas a preguntas

2.2 Capacidad de modificar y justificar código en revisión

4. Extra

- **Puntos extra:** verificación opcional del flujo con **Dapr** (`/dapr-#carnet`).